

FINAL JAVA LAST 5-MIN REVISION

1. JVM vs JRE vs JDK

Component Purpose

JVM	Runs bytecode
JRE	JVM + libraries
JDK	JRE + compiler/tools

✓ Compile/debug/run Java → **JDK**

2. Data Types

Primitive Types

byte
short
int
long
float
double
char
boolean

Byte Range

-128 to 127

Overflow example:

```
byte b = 127;  
b++;  
System.out.println(b);
```

Output:

-128

3. Type Casting

Widening

```
int a = 10;  
double b = a;
```

Narrowing

```
double a = 10.5;  
int b = (int)a;
```

4. OOP

Class

Blueprint of object.

Object

Instance of class.

5. Constructor

- Same name as class
- No return type
- Executes automatically

```
class Test{  
    Test(){  
        System.out.println("Constructor");  
    }  
}
```

6. this Keyword

Refers to current object.

```
this.a = a;
```

7. super Keyword

Calls parent constructor/variable.

```
super();
```

8. Inheritance

```
class A{}  
class B extends A{}
```

Types:

Single

Multilevel

Hierarchical

(Java does NOT support multiple inheritance using classes)

9. Polymorphism

Compile Time

Method overloading.

Runtime

Method overriding.

10. Method Overloading

Same method name, different parameters.

```
add(int a,int b)  
add(int a,int b,int c)
```

11. Method Overriding

Same method in child class.

```
class A{  
    void show(){}  
}
```

```
class B extends A{  
    void show(){  
    }  
}
```

12. Abstraction

Hides implementation details.

Abstract Class

- Cannot create object
- Can contain abstract + normal methods

```
abstract class A{  
    abstract void show();  
}
```

✅ Relies on subclasses for implementation.

13. Interface

- Fully abstract (before Java 8)
- Supports multiple inheritance

```
interface A{  
    void show();  
}
```

Keyword:

implements

14. Final Keyword

final variable

Cannot change value.

final method

Cannot override.

final class

Cannot inherit.

15. Sealed Class

Restricts inheritance.

sealed class A permits B,C{}

16. Encapsulation

Data hiding using:

private variables + getter/setter

Properties implemented using:

☒ Methods

17. Strings

String Pool

```
String s1 = "java";
```

```
String s2 = "java";
```

```
s1 == s2
```

Output:

```
true
```

new String()

```
String s1 = "java";
```

```
String s2 = new String("java");
```

```
s1 == s2
```

Output:

false

s1.equals(s2)

Output:

true

18. Important String Outputs

System.out.println(10 + 20 + "Java");

Output:

30Java

System.out.println("Java" + 10 + 20);

Output:

Java1020

19. Exception Handling

Keywords

Keyword Purpose

try risky code

catch handles exception

finally always executes

throw explicitly throw exception

throws declares exception

20. Common Exceptions

Exception	Cause
ArithmeticException	divide by zero
NullPointerException	null object
ArrayIndexOutOfBoundsException	invalid index
NumberFormatException	invalid conversion
IOException	input/output error

21. Checked vs Unchecked

Checked

Compile time.

Examples:

IOException

SQLException

FileNotFoundException

Unchecked

Runtime.

Examples:

ArithmeticException

NullPointerException

22. InputStream read()

read() throws IOException

✅ Answer:

IOException

23. Threads

Creating Thread

Extend Thread

```
class A extends Thread{}
```

Implement Runnable

```
class A implements Runnable{
```

24. run() vs start()

run()

Normal method call.

start()

Creates new thread.

```
thread.run();
```

Output thread:

main

25. Collections

List

Ordered, duplicates allowed.

Set

No duplicates.

Map

Key-value pairs.

26. Array Basics

```
int[] arr = {10,20,30};
```

```
System.out.println(arr[1]);
```

Output:

20

27. Important Keywords

Keyword	Use
static	class level
final	constant/restrict
this	current object
super	parent class
extends	inheritance
implements	interface

28. Access Modifiers

Modifier	Access
private	same class
default	same package
protected	package + subclass
public	everywhere

29. Static Keyword

Belongs to class.

static int count;

Static method:

public static void main(String[] args)

30. MOST IMPORTANT MCQs

✓ Java is:

Platform Independent

✓ Object superclass:

Object class

✓ Multiple inheritance through:

Interface

✓ Java follows:

OOP

✓ Array index starts from:

0

✓ Binary search complexity:

$O(\log n)$

✓ Stack:

LIFO

✓ Queue:

FIFO

✓ `Instant.now()`

UTC time

✓ `throw`

Explicitly raises exception

SUPER IMPORTANT OUTPUTS

1

```
System.out.println(5 + 5 + "5");
```

Output:

105

2

```
System.out.println("5" + 5 + 5);
```

Output:

555

3

```
int a = 10/0;
```

Exception:

ArithmeticException

4

```
String s = null;
```

```
s.length();
```

Exception:

NullPointerException

FINAL MEMORY SHORTCUTS

JDK = Development

JRE = Runtime

JVM = Execution

start() = new thread

run() = normal method

== compares reference
equals() compares content

throw = raise
throws = declare

Abstract class = partial abstraction
Interface = full abstraction

JDK < JRE < part???? comper loader? engine??

JVM / JRE / JDK FINAL STRUCTURE ⚡

1. JVM (Java Virtual Machine)

Main job:

Runs Java bytecode

Inside JVM:

Class Loader

Memory Area

Execution Engine

Garbage Collector

JVM FLOW

.java



Compiler (javac)



.class (bytecode)



JVM runs it

2. Class Loader ⚡

Loads .class files into memory.

Types:

Bootstrap
Extension
Application

Work:

- Loads classes
- Verifies bytecode
- Links classes

3. Memory Areas in JVM

Area	Stores
Heap	Objects
Stack	Method calls/local variables
Method Area	Class metadata/static data
PC Register	Current instruction
Native Stack	Native methods

IMPORTANT MCQ:

☒ Objects stored in:

Heap

☒ Method calls stored in:

Stack

4. Execution Engine ⚡

Executes bytecode.

Contains:

Interpreter
JIT Compiler
Garbage Collector

Interpreter

Executes line by line.

Slower.

JIT Compiler (Just In Time)

Converts bytecode → machine code.

Makes Java faster.

VERY IMPORTANT MCQ.

5. Garbage Collector ⚡

Automatically removes unused objects from heap memory.

Example:

```
Student s = new Student();  
s = null;
```

Old object becomes eligible for GC.

6. JRE (Java Runtime Environment)

Contains:

JVM + Libraries

Used ONLY to run Java programs.

NO compiler.

7. JDK (Java Development Kit)

Contains:

JRE + Development Tools

Tools:

javac

javadoc

jar

debugger

Used to:

Compile + Run + Debug

COMPLETE HIERARCHY ⚡

JDK

└─ JRE

└─ JVM

OR:

$$JDK \supset JRE \supset JVM$$

Meaning:

- JDK contains JRE
 - JRE contains JVM
-

SUPER IMPORTANT ONE-LINERS

✅ JVM:

Executes bytecode

✅ JRE:

Provides runtime environment

✅ JDK:

Provides development tools

✓ Heap:

Stores objects

✓ Stack:

Stores method calls

✓ JIT:

Improves performance

✓ Garbage Collector:

Destroys unused objects

LAST EXAM MEMORY TRICK ⚡

JDK → Develop

JRE → Run

JVM → Execute

Heap → Objects

Stack → Methods

JIT → Speed

GC → Cleanup

1. Wrapper Classes

Integer

Double

Character

2. Autoboxing / Unboxing

Integer x = 10;

int y = x;

3. File Handling basics

FileReader

BufferedReader

4. Serialization

Serializable

Externalizable

5. Marker Interface

Serializable

Cloneable

6. Lambda Expression

()->{}